Training Algorithms

Thanh-Sach LE

✉ ltsach@hcmut.edu.vn

Faculty of Computer Science & Engineering
Ho Chi Minh City University of Technology (HCMUT), VNU-HCM

Tp.HCM on February 5, 2026

Content

ltsach@hcmut.edu.vn
Thanh-Sach LE

Problem Setup

ltsach@hcmut.edu.vn
Thanh-Sach LE

Training Setup

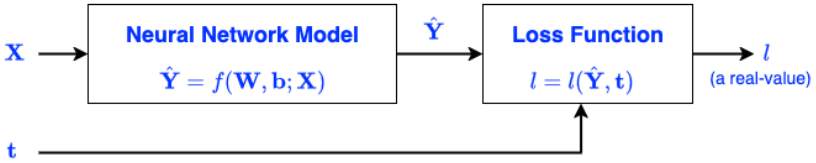
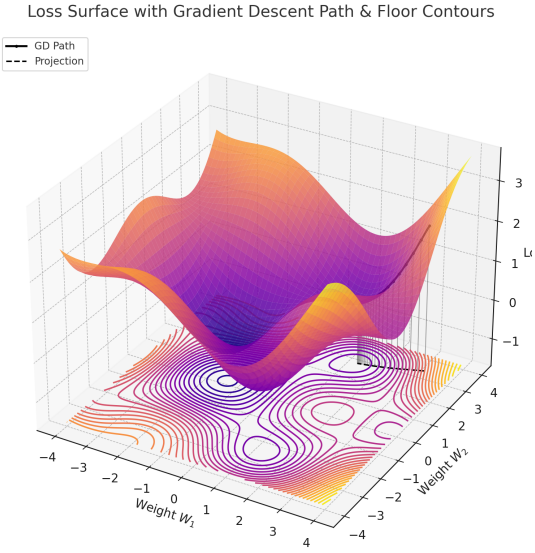


Figure 1.1: Training diagram

- Goal: learn parameters $\theta = \{\mathbf{W}, \mathbf{b}\}$ to minimize a loss function.
- Typical losses:
  - Regression: Mean Squared Error (MSE).
  - Classification: Binary Cross-Entropy (BCE) or Categorical Cross-Entropy.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Loss Surface



ltsach@hcmut.edu.vn
Thanh-Sach LE

Regression Losses

Notation: dataset $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^n$, prediction $\hat{y}_i = f_{\boldsymbol{\theta}}(\boldsymbol{x}_i)$.

Mean Squared Error (MSE)

$$\mathcal{L}_{\text{MSE}} = \frac{1}{n} \sum_{i=1}^{n} \left(y_i - \hat{y}_i\right)^2$$

Meaning: penalizes squared error, amplifies large mistakes.
Note: sensitive to outliers; suitable if noise $\sim \mathcal{N}(0, \sigma^2)$.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Regression Losses

Notation: dataset $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^n$, prediction $\hat{y}_i = f_{\boldsymbol{\theta}}(\boldsymbol{x}_i)$.

Mean Absolute Error (MAE)

$$\mathcal{L}_{\text{MAE}} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

Meaning: measures absolute error, more robust to outliers.
Note: not differentiable at 0; may converge slower than MSE.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Regression Losses

Notation: dataset $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^n$, prediction $\hat{y}_i = f_{\boldsymbol{\theta}}(\boldsymbol{x}_i)$.

Huber Loss

$$\mathcal{L}_\delta(e_i) = \begin{cases} \frac{1}{2}e_i^2 & \text{if } |e_i| \leq \delta \\ \delta(|e_i| - \frac{1}{2}\delta) & \text{if } |e_i| > \delta \end{cases} \quad \text{with } e_i = y_i - \hat{y}_i$$

Meaning: combines MSE (small errors) and MAE (large errors).
Note: hyperparameter δ controls sensitivity to outliers.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Binary Classification Loss

Notation: label $y_i \in \{0, 1\}$, logit $z_i \in \mathbb{R}$, probability $p_i = \sigma(z_i) = \frac{1}{1+e^{-z_i}}$.

Binary Cross-Entropy (BCE)

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{n} \sum_{i=1}^{n} \Big[y_i \log p_i + (1 - y_i) \log(1 - p_i) \Big]$$

Meaning: maximizes Bernoulli likelihood; penalizes confident but wrong predictions.

Note: use stable *logits version* to avoid overflow:

$$\mathcal{L}_{\text{BCE-Logits}} = \frac{1}{n} \sum_{i=1}^{n} \big(\max(0, z_i) - y_i z_i + \log(1 + e^{-|z_i|}) \big)$$

ltsach@hcmut.edu.vn
Thanh-Sach LE

Binary Classification Loss

Notation: label $y_i \in \{0, 1\}$, logit $z_i \in \mathbb{R}$, probability $p_i = \sigma(z_i) = \frac{1}{1+e^{-z_i}}$.

- Class imbalance: apply class weights or `pos_weight`.
- Threshold: default 0.5, may tune via ROC/PR.
- Regularization (L2, dropout) helps prevent overfitting.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Multiclass Classification Loss

Notation: K classes, one-hot label $\mathbf{y}_i \in \{0,1\}^K$, logits $\mathbf{z}_i \in \mathbb{R}^K$, probabilities $p_{ik} = \text{softmax}(\mathbf{z}_i)_k$.

Categorical Cross-Entropy (CE)

$$\mathcal{L}_{\text{CE}} = -\frac{1}{n}\sum_{i=1}^{n}\sum_{k=1}^{K} y_{ik}\,\log p_{ik} = -\frac{1}{n}\sum_{i=1}^{n}\log p_{i,\,y_i}$$

Meaning: maximizes Multinoulli likelihood.
Note: always use *logits version* (softmax+CE combined) for stability.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Multiclass Classification Loss

Notation: K classes, one-hot label $\mathbf{y}_i \in \{0,1\}^K$, logits $\mathbf{z}_i \in \mathbb{R}^K$, probabilities $p_{ik} = \text{softmax}(\mathbf{z}_i)_k$.

Label Smoothing

Replace one-hot: $y_{i,y_i} = 1 - \varepsilon$, others $y_{ik} = \frac{\varepsilon}{K-1}$.
Meaning: prevents overconfidence, improves generalization.

Focal Loss

$$\mathcal{L}_{\text{Focal}} = -\frac{1}{n} \sum_{i=1}^{n} \alpha_{y_i} (1 - p_{i,y_i})^{\gamma} \log p_{i,y_i}$$

Meaning: focuses on hard examples (p_{i,y_i} small).
Note: parameters $\gamma \geq 0$ (focusing) and α_k (class balance).

ltsach@hcmut.edu.vn
Thanh-Sach LE

Training Process

ltsach@hcmut.edu.vn
Thanh-Sach LE

Training Process: Overview

- **Goal:** optimize parameters $\boldsymbol{\theta} = \{\boldsymbol{W}, \boldsymbol{b}\}$ to minimize loss $\mathcal{L}$.
- Direct minimization is infeasible due to:
  - Non-linear models (e.g., deep networks).
  - High-dimensional parameter space.
- Solution: iterative procedure based on gradients.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Why Three Steps?

- **Forward pass:** compute predictions $\hat{y}$ and evaluate loss $\mathcal{L}$.
- **Backward pass:** compute gradients $\nabla_{\theta}\mathcal{L}$ via backpropagation.
- **Update step:** adjust parameters $\boldsymbol{\theta}$ using an optimizer (e.g., SGD, Adam).

Key Idea

Splitting into 3 steps ensures:

- Clear separation of computation and optimization.
- Efficient reuse of intermediate activations.
- Scalable training for large models.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Forward Pass

- Input vector: $\mathbf{x}_i$
- Prediction: $\hat{y}_i = f_{\boldsymbol{\theta}}(\boldsymbol{x}_i)$
- Loss function: $\mathcal{L} = \frac{1}{n} \sum_{i=1}^{n} \ell(y_i, \hat{y}_i)$

Purpose

- Compute outputs from current parameters.
- Measure discrepancy between prediction and ground truth.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Chain Rule and Backpropagation

Key Idea

Gradients flow **backward** through the computational graph. For each parameter w on the path from input to loss:

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_1} \cdot \frac{\partial z_1}{\partial z_2} \cdot \cdots \cdot \frac{\partial z_k}{\partial w}$$

Each term is a **local derivative**; the chain rule multiplies them along the path.

- Frameworks (PyTorch, TensorFlow) compute this automatically via autograd.
- Understanding the idea helps debug gradients and design custom layers.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Backward Pass

- Goal: compute gradients $\nabla_{\boldsymbol{\theta}} \mathcal{L}$.
- Use chain rule through computational graph (backpropagation).

Example

For parameter $w \in \mathbf{W}$:

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w}$$

- Requires saving intermediate activations during forward pass.
- Gradients indicate "direction of steepest ascent" of loss.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Update Step

- Parameters are updated using optimizer.
- **Stochastic Gradient Descent (SGD):**

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} \mathcal{L}$$

- **Adam (adaptive method):**

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\nabla_{\boldsymbol{\theta}}\mathcal{L}, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2)(\nabla_{\boldsymbol{\theta}}\mathcal{L})^2$$

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \frac{m_t}{\sqrt{v_t}+\epsilon}$$

Purpose

Iteratively refine $\boldsymbol{\theta}$ until convergence (loss minimized).

ltsach@hcmut.edu.vn
Thanh-Sach LE

Example

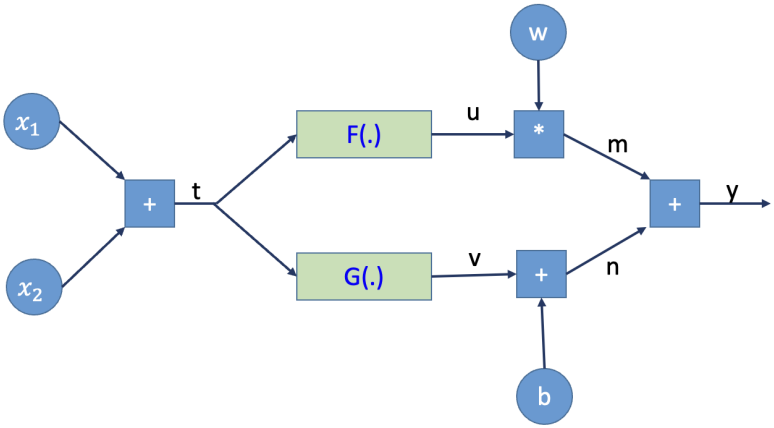


Figure 2.1: A Computational Graph

ltsach@hcmut.edu.vn
Thanh-Sach LE

Example

Forward

- Addition node: $t = x_1 + x_2$
  - Local forward: $t = x_1 + x_2$
  - Local backward: $\frac{\partial t}{\partial x_1} = 1$
  - Local backward: $\frac{\partial t}{\partial x_2} = 1$
  - Cache: None
- F(.) node:
  - Local forward: $u = F(t)$
  - Local backward: $\frac{\partial u}{\partial t} = \frac{\partial F}{\partial t} = F'(t)$
  - Cache: None
- G(.) node:
  - Local forward: $v = G(t)$
  - Local backward: $\frac{\partial v}{\partial t} = \frac{\partial G}{\partial t} = G'(t)$
  - Cache: None

ltsach@hcmut.edu.vn
Thanh-Sach LE

Example

Forward

- Multiplication node: $m = wu$
  - Local forward: $m = wu$
  - Local backward: $\frac{\partial m}{\partial w} = u$
  - Local backward: $\frac{\partial m}{\partial u} = w$
  - Cache: u
- Addition node: $n = v + b$
  - Local forward: $n = v + b$
  - Local backward: $\frac{\partial n}{\partial v} = 1$
  - Local backward: $\frac{\partial n}{\partial b} = 1$
  - Cache: None
- Addition node: $y = m + n$
  - Local forward: $y = m + n$
  - Local backward: $\frac{\partial y}{\partial m} = 1$
  - Local backward: $\frac{\partial y}{\partial n} = 1$
  - Cache: None

ltsach@hcmut.edu.vn
Thanh-Sach LE

Example

Backward to w and b

- $\frac{\partial y}{\partial w} = \frac{\partial y}{\partial m} \cdot \frac{\partial m}{\partial w} = 1 \cdot u = u$
- $\frac{\partial y}{\partial b} = \frac{\partial y}{\partial n} \cdot \frac{\partial n}{\partial b} = 1 \cdot 1 = 1$

Backward to x_1 and x_2

- $\frac{\partial y}{\partial x_1} = \left(\frac{\partial y}{\partial m} \cdot \frac{\partial m}{\partial u} \cdot \frac{\partial u}{\partial t} + \frac{\partial y}{\partial n} \cdot \frac{\partial n}{\partial v} \cdot \frac{\partial v}{\partial t}\right) \cdot \frac{\partial t}{\partial x_1}$
- $= (1 \cdot w \cdot F'(t) + 1 \cdot 1 \cdot G'(t)) \cdot 1 = w \cdot F'(t) + G'(t)$
- $\frac{\partial y}{\partial x_2} = \left(\frac{\partial y}{\partial m} \cdot \frac{\partial m}{\partial u} \cdot \frac{\partial u}{\partial t} + \frac{\partial y}{\partial n} \cdot \frac{\partial n}{\partial v} \cdot \frac{\partial v}{\partial t}\right) \cdot \frac{\partial t}{\partial x_2}$
- $= (1 \cdot 1 \cdot F'(t) + 1 \cdot 1 \cdot G'(t)) \cdot 1 = F'(t) + G'(t)$

ltsach@hcmut.edu.vn
Thanh-Sach LE

Pseudocode: SGD Training Algorithm

Inputs

Dataset $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^{n}$, model $f_{\boldsymbol{\theta}}$, loss $\ell(\cdot, \cdot)$, learning rate η, epochs E, mini-batch size B.

1. Initialize parameters $\boldsymbol{\theta}$.
2. **for** epoch $= 1$ to E **do**
   1. Shuffle the dataset and split into mini-batches of size B.
   2. **for** each mini-batch $(\mathbf{X}, \mathbf{y})$ **do**
      1. *Forward:* $\hat{\boldsymbol{Y}} \leftarrow f_{\boldsymbol{\theta}}(\boldsymbol{X})$
      2. *Loss:* $\mathcal{L} \leftarrow \ell(\mathbf{y}, \hat{\mathbf{y}})$
      3. *Backward:* compute gradients $\nabla_{\boldsymbol{\theta}} \mathcal{L}$
      4. *Update (SGD):* $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} \mathcal{L}$
   3. **end for**
3. **end for**

ltsach@hcmut.edu.vn
Thanh-Sach LE

Notes on Training Terminology

Key Concepts

- **Mini-batch:** a small subset of the training dataset used in one forward–backward–update cycle. Balances efficiency and gradient stability.
- **Epoch:** one complete pass through the entire training dataset (all mini-batches).
- **Iteration (step):** one update of parameters using a single mini-batch.
- **Learning rate η:** step size for parameter updates; too large $\Rightarrow$ divergence, too small $\Rightarrow$ slow convergence.
- **Momentum:** technique to smooth updates by accumulating past gradients, reducing oscillations.
- **Learning rate schedule:** strategy to adjust η during training (e.g., decay, cosine annealing).

ltsach@hcmut.edu.vn
Thanh-Sach LE

Optimization: Common Variants

ltsach@hcmut.edu.vn
Thanh-Sach LE

Optimization Landscape

Method	Update (sketch)	Hyperparams	Pros	Cons / Typical Use
SGD + Momentum	$\mathbf{v}_t = \mu \mathbf{v}_{t-1} + \nabla_{\boldsymbol{\theta}} \mathcal{L}$, $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \mathbf{v}_t$	η, μ	Simple, strong baselines; good generalization	Sensitive to LR; may oscillate; vision/classic DL
Nesterov (NAG)	$\nabla_{\boldsymbol{\theta} - \eta\mu \mathbf{v}_{t-1}} \mathcal{L}$ (lookahead gradient)	η, μ	Faster than momentum in convex; anticipates step	Extra grad eval; tuning needed
AdaGrad	$\mathbf{G}_t = \mathbf{G}_{t-1} + \mathbf{g}_t^2$; $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \mathbf{g}_t / (\sqrt{\mathbf{G}_t} + \epsilon)$	η, ϵ	Per-parameter adaptivity; sparse data friendly	LR decays too much; NLP (sparse)
RMSProp	$\mathbf{s}_t = \rho \mathbf{s}_{t-1} + (1-\rho)\mathbf{g}_t^2$; $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \mathbf{g}_t / (\sqrt{\mathbf{s}_t} + \epsilon)$	η, ρ, ϵ	Fixes AdaGrad decay; stable training	Needs schedule; widely used in RNNs
Adam	$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\mathbf{g}_t$; $\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1-\beta_2)\mathbf{g}_t^2$; $\hat{\mathbf{m}}_t = \mathbf{m}_t / (1-\beta_1^t)$; $\hat{\mathbf{v}}_t = \mathbf{v}_t / (1-\beta_2^t)$; $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \hat{\mathbf{m}}_t / (\sqrt{\hat{\mathbf{v}}_t} + \epsilon)$	η, β_1, β_2, ϵ	Fast convergence; default in many tasks	Can overfit; needs weight decay; careful LR
AdamW	Adam + decoupled weight decay: $\boldsymbol{\theta} \leftarrow (1 - \eta\lambda)\boldsymbol{\theta} - \eta \hat{\mathbf{m}}_t / (\sqrt{\hat{\mathbf{v}}_t} + \epsilon)$	η, β_1, β_2, ϵ, λ	Better generalization than Adam; modern default	Adds decay hyperparam; transformers/vision
Nadam	Adam + Nesterov momentum (on $\mathbf{m}_t$)	Adam params	Often slightly faster than Adam	Small practical gains; more complex
Adafactor	Factorized 2nd-moment for matrices (memory saving)	η (often schedule)	Low memory; large models (T5)	Fewer guarantees; sensitive schedules
LAMB	AdamW + trust ratio (layer-wise scaling)	AdamW + trust ratio	Large-batch training (1K–64K)	Heavier tuning; large-scale pretraining
SAM	Minimize sharpness: update toward flat minima	Base opt + ρ	Better generalization; robust minima	Extra forward/backward per step; vision/NLP
Lion	Sign-based momentum; update uses $\text{sign}(\mathbf{m}_t)$	η, β_1, β_2	Memory-light; strong on vision	Newer; tuning may vary by task

Notes

Popularity: Adam/AdamW → default in modern practice; SGD+Momentum still strong in vision.
Factors: batch size, weight decay, schedule (cosine, warmup), memory budget, data sparsity.

ltsach@hcmut.edu.vn
Thanh-Sach LE

SGD with Momentum — Update

Formula

$$\mathbf{g}_t = \nabla_{\boldsymbol{\theta}} \mathcal{L}_t(\boldsymbol{\theta}), \quad \mathbf{v}_t = \mu \mathbf{v}_{t-1} + \mathbf{g}_t, \quad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta\, \mathbf{v}_t$$

- $\boldsymbol{\theta}$: parameter vector (e.g., $\{\boldsymbol{W}, \boldsymbol{b}\}$).
- $\mathbf{g}_t$: gradient on current mini-batch.
- $\mathbf{v}_t$: momentum (velocity) vector.
- **Initialization:** set $\mathbf{v}_0 = \mathbf{0}$ at the beginning of training.

Note

$\mathbf{v}_t$ is recursive: each update depends on previous momentum $\mathbf{v}_{t-1}$. Initialization with zero is standard; after several steps, $\mathbf{v}_t$ accumulates a smoothed history of past gradients.

ltsach@hcmut.edu.vn
Thanh-Sach LE

SGD with Momentum — Hyperparameters

Terms

- μ: momentum coefficient, scalar. Typical range: 0.8–0.99 (commonly 0.9).
- η: learning rate, scalar. Task-dependent; try 10^{-3}–10^{-1} with schedule (cosine, step).
- λ: weight decay (optional). Often 10^{-4} in CNN training from scratch.

ltsach@hcmut.edu.vn
Thanh-Sach LE

SGD with Momentum — Intuition

Why Momentum?

- Accumulates a running direction (inertia).
- Dampens oscillations across steep directions.
- Accelerates along shallow valleys.
- More stable than vanilla SGD in ill-conditioned landscapes.

ltsach@hcmut.edu.vn
Thanh-Sach LE

SGD with Momentum — Usage

Practical Notes

- Strong baseline for CNNs; often generalizes better than adaptive methods.
- Pair with cosine decay + warmup for learning rate.
- Tune η first, then adjust μ and λ.
- Scale η with batch size (linear rule as heuristic).

ltsach@hcmut.edu.vn
Thanh-Sach LE

Nesterov Accelerated Gradient (NAG) — Update

Formula

$$\mathbf{g}_t = \nabla_{\boldsymbol{\theta}} \mathcal{L}\big(\boldsymbol{\theta} - \eta\mu\mathbf{v}_{t-1}\big), \quad \mathbf{v}_t = \mu\mathbf{v}_{t-1} + \mathbf{g}_t, \quad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta\,\mathbf{v}_t$$

- $\boldsymbol{\theta}$: parameter vector.
- $\mathbf{g}_t$: gradient evaluated at the *lookahead* position.
- $\mathbf{v}_t$: momentum (velocity) vector.
- **Initialization:** set $\mathbf{v}_0 = \mathbf{0}$.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Nesterov Accelerated Gradient (NAG) — Hyperparameters

Terms

- μ: momentum coefficient, scalar. Typical range: 0.8–0.99 (commonly 0.9).
- η: learning rate, scalar. Similar scale as in SGD with momentum; tune carefully.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Nesterov Accelerated Gradient (NAG) — Intuition

Why Nesterov?

- Anticipates the effect of the momentum step (lookahead).
- Provides a correction before moving, often leading to faster convergence.
- Reduces overshoot compared to classical momentum.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Nesterov Accelerated Gradient (NAG) — Usage

Practical Notes

- Often faster than vanilla momentum in convex and smooth settings.
- More sensitive to learning rate η; careful tuning needed.
- Used in vision tasks and sometimes in RNN training.
- Common default: $\mu = 0.9$, cosine LR decay with warmup.

ltsach@hcmut.edu.vn
Thanh-Sach LE

AdaGrad — Update

Formula

$$\mathbf{G}_t = \mathbf{G}_{t-1} + \mathbf{g}_t^2, \quad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \frac{\mathbf{g}_t}{\sqrt{\mathbf{G}_t} + \epsilon}$$

- $\boldsymbol{\theta}$: parameter vector.
- $\mathbf{g}_t$: gradient at step t (element-wise squared in $\mathbf{g}_t^2$).
- $\mathbf{G}_t$: accumulated sum of squared gradients (per parameter).
- **Initialization:** set $\mathbf{G}_0 = \mathbf{0}$.

ltsach@hcmut.edu.vn
Thanh-Sach LE

AdaGrad — Hyperparameters

Terms

- η: global learning rate (scalar). Typical start: 0.01 or 0.1.
- ϵ: small constant for numerical stability, e.g. 10^{-8}.

ltsach@hcmut.edu.vn
Thanh-Sach LE

AdaGrad — Intuition

Why AdaGrad?

- Adapts learning rate for each parameter individually.
- Frequent features $\Rightarrow$ smaller effective step size. Infrequent features $\Rightarrow$ larger effective step size.
- Naturally well-suited for sparse data (e.g., text / NLP).

ltsach@hcmut.edu.vn
Thanh-Sach LE

AdaGrad — Usage

Practical Notes

- Good choice for sparse features or embeddings.
- Main drawback: effective learning rate decays monotonically, can stall long training.
- Rarely used for deep networks today; mostly replaced by RMSProp/Adam.
- Still relevant in NLP and recommendation tasks with sparse gradients.

ltsach@hcmut.edu.vn
Thanh-Sach LE

RMSProp — Update

Formula

$$\mathbf{s}_t = \rho \mathbf{s}_{t-1} + (1-\rho)\mathbf{g}_t^2, \quad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \frac{\mathbf{g}_t}{\sqrt{\mathbf{s}_t} + \epsilon}$$

- $\boldsymbol{\theta}$: parameter vector.
- $\mathbf{g}_t$: gradient at step t.
- $\mathbf{s}_t$: exponentially decayed average of squared gradients.
- **Initialization:** set $\mathbf{s}_0 = \mathbf{0}$.

ltsach@hcmut.edu.vn
Thanh-Sach LE

RMSProp — Hyperparameters

Terms

- η: learning rate (scalar). Typical range: 10^{-4} – 10^{-2}.
- ρ: decay rate (forgetting factor). Commonly 0.9.
- ϵ: small constant for stability, e.g. 10^{-8}.

ltsach@hcmut.edu.vn
Thanh-Sach LE

RMSProp — Intuition

Why RMSProp?

- Fixes AdaGrad's issue of monotonically decaying learning rates.
- Maintains a moving average of squared gradients.
- Effectively adapts learning rate per parameter in non-stationary problems.
- Provides more stable training for deep neural nets.

ltsach@hcmut.edu.vn
Thanh-Sach LE

RMSProp — Usage

Practical Notes

- Well-suited for non-stationary objectives (e.g., RNN training).
- Popular choice before Adam; still used in reinforcement learning.
- Tune η and ρ carefully; defaults often $\eta = 0.001$, $\rho = 0.9$.
- Consider combining with momentum for improved stability.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Adam — Update

Formula

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\mathbf{g}_t, \quad \mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1-\beta_2)\mathbf{g}_t^2$$

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1-\beta_1^t}, \quad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1-\beta_2^t}, \quad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon}$$

- $\boldsymbol{\theta}$: parameter vector.
- $\mathbf{g}_t$: gradient at step t.
- $\mathbf{m}_t$: exponential moving average of gradients (first moment).
- $\mathbf{v}_t$: exponential moving average of squared gradients (second moment).
- $\hat{\mathbf{m}}_t, \hat{\mathbf{v}}_t$: bias-corrected estimates.
- **Initialization:** $\mathbf{m}_0 = \mathbf{0}, \mathbf{v}_0 = \mathbf{0}$.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Adam — Hyperparameters

Typical Settings

- η: learning rate (default 0.001).
- β_1: decay for first moment (default 0.9).
- β_2: decay for second moment (default 0.999).
- ϵ: stability constant (default 10^{-8}).

ltsach@hcmut.edu.vn
Thanh-Sach LE

Adam — Intuition

Why Adam?

- Combines benefits of Momentum (first moment) and RMSProp (second moment).
- Adaptive per-parameter learning rates.
- Bias-correction ensures stability in early iterations.
- Usually fast and robust across many tasks.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Adam — Usage

Practical Notes

- **Default optimizer in many frameworks (PyTorch, TensorFlow)**.
- Works well out-of-the-box for NLP and RL tasks.
- Can overfit or lead to poor generalization without weight decay.
- Often combined with learning rate schedules (cosine decay, warmup).
- For vision models (CNNs, transformers), AdamW (decoupled weight decay) is preferred.

ltsach@hcmut.edu.vn
Thanh-Sach LE

AdamW — Update

Formula

$$\boldsymbol{\theta} \leftarrow (1 - \eta\lambda)\,\boldsymbol{\theta} \; - \; \eta\, \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon}$$

- $\hat{\mathbf{m}}_t$: bias-corrected first moment estimate.
- $\hat{\mathbf{v}}_t$: bias-corrected second moment estimate.
- η: learning rate.
- λ: weight decay coefficient.
- **Initialization:** same as Adam ($\mathbf{m}_0 = \mathbf{0}, \mathbf{v}_0 = \mathbf{0}$).

ltsach@hcmut.edu.vn
Thanh-Sach LE

AdamW — Hyperparameters

Typical Settings

- η: default 10^{-3} (transformers often 2×10^{-4} to 5×10^{-5}).
- $\beta_1 = 0.9$, $\beta_2 = 0.999$ (same as Adam).
- $\epsilon = 10^{-8}$ for stability.
- λ: weight decay, often 0.01 for large models.

ltsach@hcmut.edu.vn
Thanh-Sach LE

AdamW — Intuition

Why AdamW?

- In Adam, adding L2 regularization inside the loss couples it with adaptive step sizes $\rightarrow$ suboptimal.
- AdamW decouples weight decay from the gradient update.
- This yields better control of regularization and significantly improves generalization.

ltsach@hcmut.edu.vn
Thanh-Sach LE

AdamW — Usage

Practical Notes

- **Default optimizer for transformers (e.g., BERT, GPT) and modern vision backbones.**
- Usually paired with cosine learning rate schedule + warmup steps.
- Tune weight decay carefully (typical 0.01, but can vary).
- Outperforms Adam in most large-scale training scenarios.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Nadam

Idea: Nesterov-style correction on Adam's momentum.
Effect: sometimes slightly faster than Adam in practice.
Tip: start from Adam's best LR/schedule; small gains, task-dependent.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Adafactor (Memory-Efficient)

Factorized second moment for matrices (approx. $\mathbf{v}_t$ by row/col stats).
Pros: drastically reduced memory; suitable for very large models.
Notes: pair with LR schedule (no fixed η); used in T5 training.

ltsach@hcmut.edu.vn
Thanh-Sach LE

LAMB (Large-Batch Training)

Trust ratio: layer-wise scaling for stable updates at huge batch sizes.
Use: pretraining with batch $>$ 1k; combine with warmup + cosine decay.
Caveat: more hyperparams; verify accuracy parity with smaller-batch baselines.

ltsach@hcmut.edu.vn
Thanh-Sach LE

SAM: Sharpness-Aware Minimization

Objective: prefer flat minima by penalizing sharpness.
Mechanism: inner ascent to worst-case in a neighborhood, then descent.
Pros: better generalization; **Cost:** $\approx 2\times$ compute per step.
Variants: ASAM, GSAM; can wrap around SGD/AdamW.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Lion (Sign-Based Momentum)

Update: uses sign($\mathbf{m}_t$) for parameter step (memory-light).
Pros: strong on vision; reduces second-moment storage.
Notes: newer; compare to AdamW on your task.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Summary & Choosing the Optimizer

- **General default:** AdamW + cosine LR + warmup; tune weight decay.
- **Vision (from-scratch CNNs):** SGD + Momentum often best generalization.
- **Large-batch pretraining:** LAMB (or AdamW + careful scaling) + warmup.
- **Sparse features / NLP classic:** AdaGrad / RMSProp still relevant.
- **Flat minima / robustness:** SAM (wrap your base optimizer).
- **Memory-limited large models:** Adafactor; **New trials:** Lion.

Practical Tips

Schedule matters (cosine, step, one-cycle); always try warmup.
Tune η first, then decay/momentum; log training curves and validate frequently.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Training Techniques

ltsach@hcmut.edu.vn
Thanh-Sach LE

Training Techniques

- **Learning rate scheduling**: step decay, cosine annealing, warm restarts.
- **Regularization**:
  - L2 (weight decay).
  - Dropout.
- **Normalization**: BatchNorm, LayerNorm.
- **Early stopping**: stop when validation error increases.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Learning Rate Scheduling

Idea

Adjust η dynamically during training to balance exploration vs. convergence.

- **Step decay**: reduce η by a factor every k epochs. Common in CNN training.
- **Cosine annealing**: $\eta_t = \frac{1}{2}\eta_0\left(1 + \cos\left(\frac{t}{T}\pi\right)\right)$. Smooth decay; often with warmup.
- **Warm restarts**: periodically reset η to a higher value $\rightarrow$ escape sharp minima.

Notes

Schedules strongly affect convergence speed and generalization.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Regularization: L2 (Weight Decay)

L2 penalty (Ridge)

Add penalty to loss:

$$\mathcal{L}_{\text{total}} = \mathcal{L} + \frac{\lambda}{2}\|\boldsymbol{\theta}\|_2^2 = \mathcal{L} + \frac{\lambda}{2}\sum_j \theta_j^2$$

- Shrinks weights toward zero; prevents large coefficients
- Equivalent to multiplicative decay in SGD: $\theta \leftarrow (1 - \eta\lambda)\theta - \eta\nabla\mathcal{L}$
- In AdamW: weight decay is decoupled from gradient; use $\lambda \approx 10^{-4}$–10^{-2}

ltsach@hcmut.edu.vn
Thanh-Sach LE

Regularization: L1 (Lasso)

L1 penalty

$$\mathcal{L}_{\text{total}} = \mathcal{L} + \lambda \|\boldsymbol{\theta}\|_1 = \mathcal{L} + \lambda \sum_j |\theta_j|$$

- Promotes **sparsity**: many weights pushed to exactly zero
- Useful for feature selection (which inputs matter)
- Less common in deep learning than L2; L2 + dropout is typical for MLPs/CNNs

ltsach@hcmut.edu.vn
Thanh-Sach LE

Regularization: Dropout

Mechanism

During training: randomly set each neuron output to zero with probability p (e.g., 0.1–0.5). At test time: use all neurons (scale by $1 - p$ if needed).

- Prevents co-adaptation: forces network to not rely on any single neuron
- Interpreted as training an ensemble of sub-networks
- Often applied after hidden layers; less common in modern transformers (LayerNorm + augmentation preferred)

ltsach@hcmut.edu.vn
Thanh-Sach LE

Normalization

Why?

Stabilize training by reducing internal covariate shift.

- **BatchNorm**: normalize activations across batch. Works well for CNNs; depends on batch size.
- **LayerNorm**: normalize across features per sample. Common in transformers; independent of batch size.

Notes

Normalization smooths the optimization landscape, allows larger η, and improves convergence.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Early Stopping

Idea

Stop training when validation error no longer decreases (to avoid overfitting).

- Monitor validation loss or accuracy each epoch.
- If no improvement for p epochs (patience), stop training.
- Save best model weights (checkpointing).

Notes

Simple and effective; acts as a form of implicit regularization.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Practical Considerations

ltsach@hcmut.edu.vn
Thanh-Sach LE

Practical Considerations — Initialization

Initialization

- Proper weight initialization stabilizes variance across layers.
- **Xavier (Glorot)**: suitable for sigmoid/tanh activations.
- **He (Kaiming)**: designed for ReLU and variants.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Practical Considerations — Gradients

Vanishing Gradients

- Gradients shrink as they backprop through deep networks.
- Mitigation: ReLU activations, residual connections, normalization layers.

Exploding Gradients

- Gradients grow uncontrollably, causing unstable updates.
- Mitigation: gradient clipping, careful initialization.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Practical Considerations — Batch Size

Batch Size Trade-off

- **Small batches:** noisier updates, slower throughput, often better generalization.
- **Large batches:** faster throughput, smoother gradients, but risk poorer generalization. Requires careful learning rate scaling (linear scaling rule).

ltsach@hcmut.edu.vn
Thanh-Sach LE

Summary

ltsach@hcmut.edu.vn
Thanh-Sach LE

Training Neural Networks: Summary

Key Points

- Training loop consists of:
  - **Forward pass:** compute predictions.
  - **Loss computation:** measure error vs. ground truth.
  - **Backward pass:** compute gradients (backpropagation).
  - **Update step:** adjust weights with optimizer.
- Choice of optimizer, learning rate schedule, and regularization strongly influences convergence and generalization.
- Validation monitoring is essential to detect overfitting and apply early stopping.

Pipeline:

Data $\rightarrow$ Forward $\rightarrow$ Loss $\rightarrow$ Backward $\rightarrow$ Update $\rightarrow$ Repeat

ltsach@hcmut.edu.vn
Thanh-Sach LE